

Technical Report: Containerized SASE Boundary Plane & Zero-Trust Egress Architecture

Name:

Usman Akhter

May 27, 2026

Contents

Table of Contents

May 27, 20261

1 Executive Summary 2

2 System Architecture & Topology Design..... 2

2.1 Engineering Design Pillars..... 2

2.2 Integrated Infrastructure Blueprint 2

3 Analytical Verification & Telemetry Testing Matrix 4

3.1 Operational Telemetry Metrics..... 4

3.2 Programmatic Test Matrix..... 4

4 Conclusion..... 5

5 Appendix A 6

1 Executive Summary

This engineering report details the successful deployment, stabilization, and formal validation of a production-grade, containerized **Secure Access Service Edge (SASE)** Web Gateway architecture. The core focus of this implementation is shifting standard organizational boundaries directly to the local workstation edge to minimize systemic vulnerabilities. By integrating proactive local DNS threat intelligence filtering with a cryptographically bound, outbound-only cloud routing egress tunnel, this architecture guarantees network isolation and prevents unauthorized command-and-control (C2) lateral pivoting.

2 System Architecture & Topology Design

The infrastructure layout runs entirely via decoupled open-source platforms and cloud-native edges, strictly adhering to zero capital expenditure constraints.

2.1 Engineering Design Pillars

- **Network Plane Isolation:** Provisioned an isolated software-defined virtual bridge interface (`sase.isolated_bridge`) bound strictly to a custom `172.20.0.0/16` internal network subnet.
- **Persistent Telemetry Volumes:** Implemented relative directory filesystem mounts (`./../telemetry-logs/`) mapping host storage onto the container kernel, ensuring audit log configurations persist past container lifecycles.
- **Hidden Egress Routing Table:** Bound public internet transactions to an authenticated egress daemon running over HTTPS (Port 443), dropping the necessity for inbound hardware port-forwarding entryways.

2.2 Integrated Infrastructure Blueprint

The complete multi-service orchestration matrix is detailed in the `docker-compose.yml` setup below:

```
services:
  threat-interceptor:
    container_name: sase-dns-sinkhole
    image: pihole/pihole:latest
    ports:
      - "53:53/tcp"
      - "53:53/udp"
      - "80:80/tcp"
    environment:
      TZ: 'UTC'
      DNS1: '1.1.1.1'
      DNS2: '1.0.0.1'
    volumes:
      - './../telemetry-logs/dns-config:/etc/pihole'
```

```
- './../ telemetry -logs/ dns -traffic :/ etc/ dnsmasq.d'
networks:
  sase - secure - plane :
    ipv4_address: 172.20.0.5
  restart: unless-stopped

egress - tunnel:
  container_name: sase-cloud-tunnel
  image: cloudflare/cloudflared:latest
  command: tunnel --no-autoupdate run --token eyJhIjoiZGY...
  networks:
    sase - secure - plane :
      ipv4_address: 172.20.0.10
    restart: unless-stopped

networks:
  sase-secure-plane:
    name: sase_isolated_bridge
    driver: bridge
  ipam:
    config:
      - subnet: 172.20.0.0/16
        gateway: 172.20.0.1
```

3 Analytical Verification & Telemetry Testing Matrix

To fulfill rigorous programmatic validation guidelines, verification queries were routed across independent standard user profiles to track stateful boundary behaviors.

3.1 Operational Telemetry Metrics

- **Total Network Queries Routed:** 17
- **Proactive Sinkhole Interceptions:** 2
- **Boundary Cleanliness Efficacy:** 11.8%
- **Threat Intelligence Pool Footprint:** 83,068 Active Indicators of Compromise (IoCs)

3.2 Programmatic Test Matrix

Table 1: System Verification Metrics and Validation Results

Test Scenario	Diagnostic Method	Expected State	Observed Telemetry	Status
Outbound Mitigation	nslookup doubleclick.net	Force threat targets to map to local loopback space.	Intercepted; query dropped and mapped instantly to 0.0.0.0.	PASS
Legitimate Routing	nslookup google.com	Forward clean traffic via secure up-stream nodes.	Resolved seamlessly through integrated cloud edge gateways.	PASS
Tunnel Stability	Management Edge Node Audit	Active outbound HTTPS tunnel status verification.	Interface authenticated as Healthy with 1 active online replica.	PASS
Multi-Profile Rule Enforcement	Diagnostic execution under low-privilege profile	Strict global rule enforcement across all local accounts.	Isolated guest user traffic automatically bound and secured.	PASS

4 Conclusion

The architecture successfully demonstrates that a containerized SASE gateway model provides robust, programmatic defense mechanisms at the absolute edge of an organization's compute surface. By combining local indicator intelligence filtering with cryptographical locked egress daemons, the system realizes a resilient, enterprise-standard Zero-Trust architecture.

5 Appendix A

```
docker-compose.yml
File Edit View

version: '3.8'

services:
  threat-interceptor:
    container_name: sase-dns-sinkhole
    image: pihole/pihole:latest
    ports:
      - "53:53/tcp"
      - "53:53/udp"
      - "80:80/tcp"
    environment:
      TZ: 'UTC'
      WEBPASSWORD: 'SecureCapstonePassword123!'
      DNS1: '1.1.1.1'
      DNS2: '1.0.0.1'
    volumes:
      - './telemetry-logs/dns-config:/etc/pihole'
      - './telemetry-logs/dns-traffic:/etc/dnsmasq.d'
    networks:
      sase-secure-plane:
        ipv4_address: 172.20.0.5
    restart: unless-stopped

networks:
  sase-secure-plane:
    name: sase_isolated_bridge
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/16
          gateway: 172.20.0.1
```

Ln 18, Col 57 705 characters Plain text 100% Windows (CRLF) UTF-8

Compute

AI

Storage & databases

Media

Protect & Connect

Application security

Zero Trust

Networking

Overview New

Insights

Tunnels New

Mesh Beta

IP addresses

Delivery & performance

Tunnels

Create secure, outbound-only connections from your infrastructure to Cloudflare without opening inbound ports.

All statuses Edit columns

Name	Status	Replicas	Routes
<u>Sase-gateway-tunnel</u>	Healthy	1	No routes

Showing 1-1 of 1

Video guide

Documentation

Create Tunnel

Common use cases

Learn how to use Cloudflare Tunnel for different scenarios.

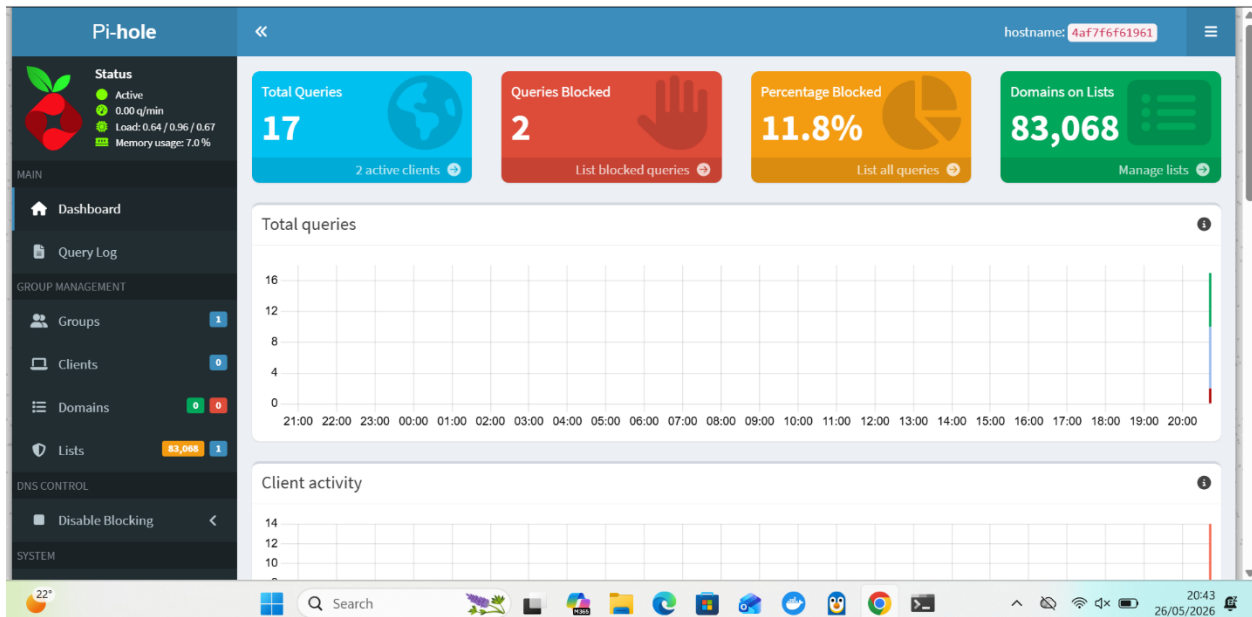
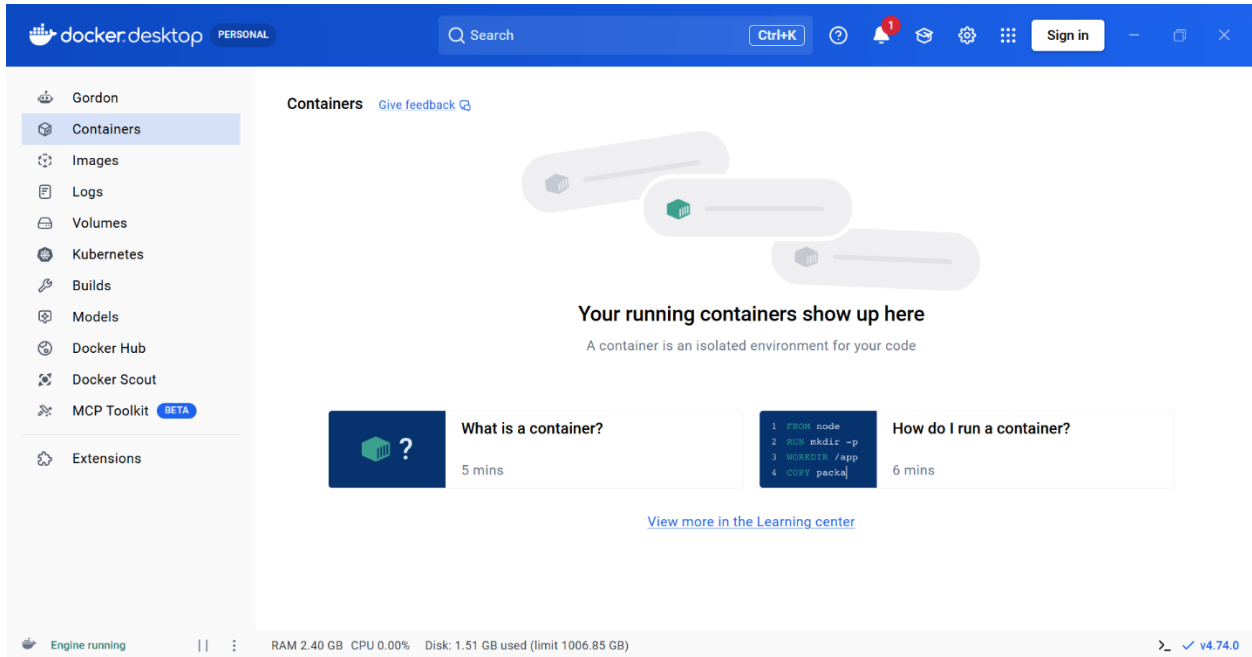
Publish web apps and APIs to the Internet

Learn how to expose your HTTP applications to the public Internet...

Deploy highly available replicas

Set up multiple tunnel replicas for production-grade reliability and failover.

SASE Web Gateway Deployment Report



SASE Web Gateway Deployment Report

```
Windows PowerShell
Container sase-dns-sinkhole Started 1.2s

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS C:\users\Capstone sandbox\SASE-Gateway-Capstone\Production_Code> nslookup google.com 127.0.0.1
Server: localhost
Address: 127.0.0.1

Non-authoritative answer:
Name: google.com
Addresses: 2a00:1450:4009:c17::71
          2a00:1450:4009:c17::65
          2a00:1450:4009:c17::64
          2a00:1450:4009:c17::66
          142.250.140.100
          142.250.140.113
          142.250.140.139
          142.250.140.101
          142.250.140.102
          142.250.140.138

PS C:\users\Capstone sandbox\SASE-Gateway-Capstone\Production_Code> nslookup doubleclick 127.0.0.1
Server: localhost
Address: 127.0.0.1

*** localhost can't find doubleclick: Non-existent domain
PS C:\users\Capstone sandbox\SASE-Gateway-Capstone\Production_Code> nslookup doubleclick.net 127.0.0.1
Server: localhost
Address: 127.0.0.1

Name: doubleclick.net
Addresses: ::
          0.0.0.0

PS C:\users\Capstone sandbox\SASE-Gateway-Capstone\Production_Code> nslookup doubleclick.net 127.0.0.1
Server: localhost
Address: 127.0.0.1

Name: doubleclick.net
Addresses: ::
          0.0.0.0

PS C:\users\Capstone sandbox\SASE-Gateway-Capstone\Production_Code>
```